

1: FORK() C PROGRAM:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

int main()

{

pid_t pid;

pid = fork();

if(pid < 0)

{

printf("Fork Failed\n");

exit(1);

}

else if(pid == 0)

{

printf("\nCHILD PROCESS");

printf("\nChild PID : %d", getpid());

printf("\nParent PID : %d\n", getppid());

exit(0);

}

else

{

wait(NULL);

printf("\nPARENT PROCESS");

printf("\nParent PID : %d", getpid());

printf("\nParent's Parent PID : %d\n", getppid());

}
```

```
return 0;
```

```
}
```

OUTPUT:

SHELL SCRIPTING:

```
#!/bin/bash
```

```
echo "Parent Process ID : $$"
```

```
(
```

```
echo "Child Process ID : $$"
```

```
echo "Parent Process ID : $PPID"
```

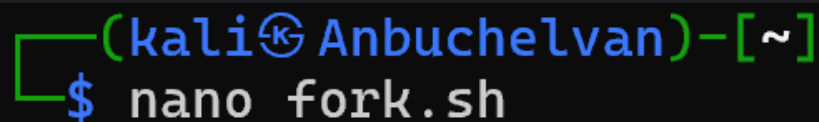
```
exit 0
```

```
) &
```

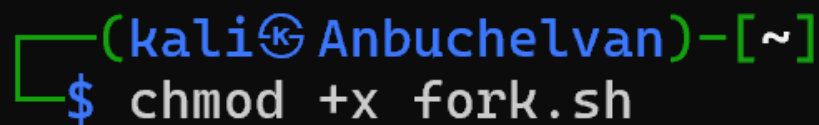
```
wait
```

```
echo "Child Process Completed"
```

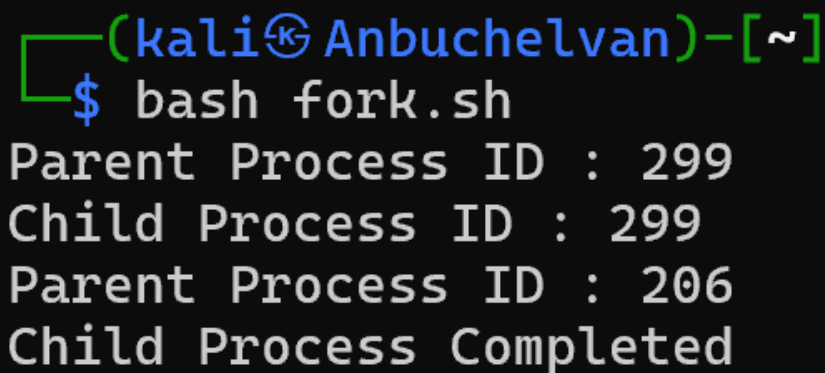
OUTPUT:



```
(kaliⓀ Anbuchelvan)-[~]  
$ nano fork.sh
```



```
(kaliⓀ Anbuchelvan)-[~]  
$ chmod +x fork.sh
```



```
(kaliⓀ Anbuchelvan)-[~]  
$ bash fork.sh  
Parent Process ID : 299  
Child Process ID : 299  
Parent Process ID : 206  
Child Process Completed
```

2: WAIT() SYSTEM CALL

C PROGRAM

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {

    pid_t pid;

    pid = fork();

    if(pid == 0)

    {

        printf("Child Process Running\n");

        sleep(5);

        printf("Child Process Completed\n");

    }

    else

    {

        wait(NULL);

        printf("Parent Resumes Execution\n");

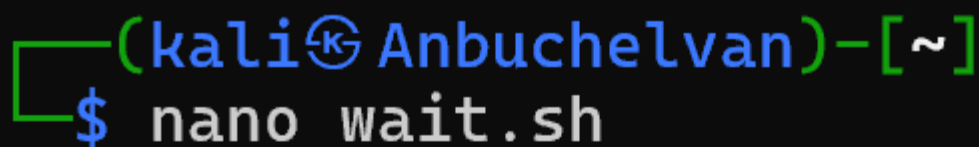
    }

    return 0; }
```

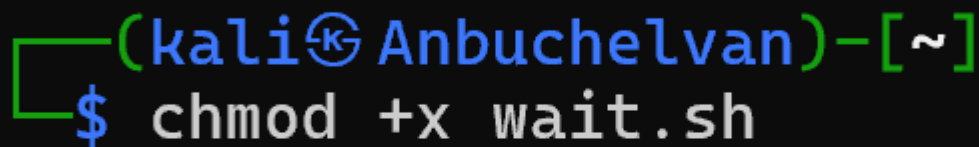
SHELL SCRIPTING:

```
#!/bin/bash (  
echo "Child Process Running"  
sleep 5  
echo "Child Process Completed"  
) &  
wait  
echo "Parent Resumes Execution"
```

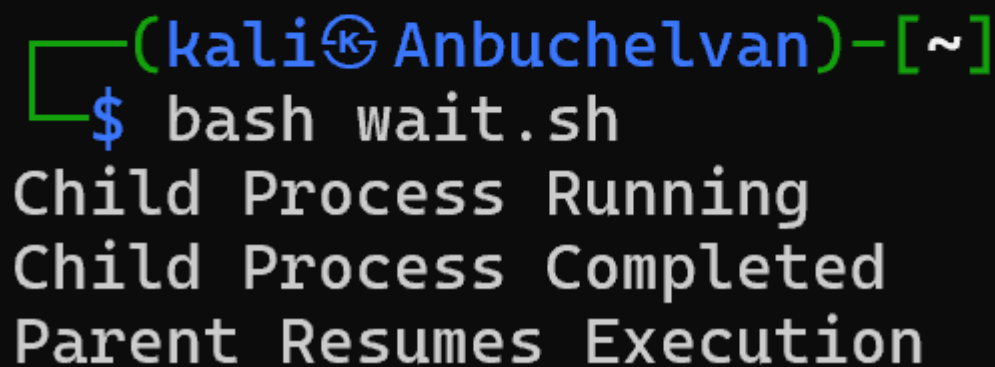
OUTPUT:



```
(kaliⓀ Anbuchelvan)-[~]  
$ nano wait.sh
```



```
(kaliⓀ Anbuchelvan)-[~]  
$ chmod +x wait.sh
```



```
(kaliⓀ Anbuchelvan)-[~]  
$ bash wait.sh  
Child Process Running  
Child Process Completed  
Parent Resumes Execution
```

3: CLOSE() SYSTEM CALL

C PROGRAM

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
int fd;
```

```
fd = open("sample.txt", O_RDONLY);
```

```
if(fd < 0)
```

```
{
```

```
printf("File Opening Failed\n");
```

```
return 1;
```

```
}
```

```
printf("File Opened Successfully\n");
```

```
close(fd);
```

```
printf("File Closed Successfully\n");
```

```
return 0;
```

```
}
```

SHELL SCRIPTING:

```
#!/bin/bash
```

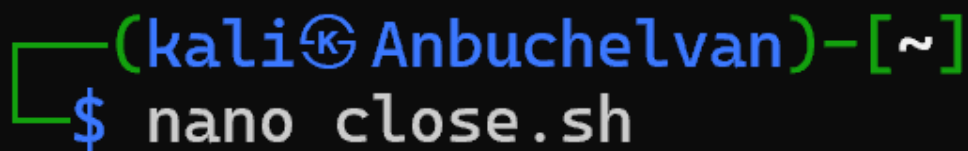
```
exec 3< sample.txt
```

```
echo "File Opened Successfully"
```

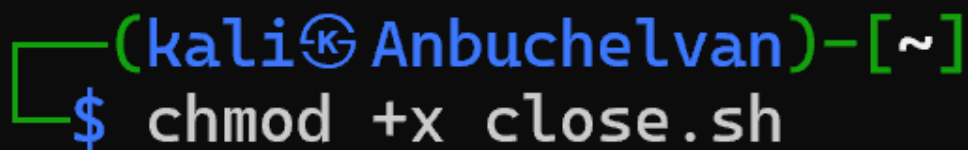
```
exec 3<&-
```

```
echo "File Closed Successfully"
```

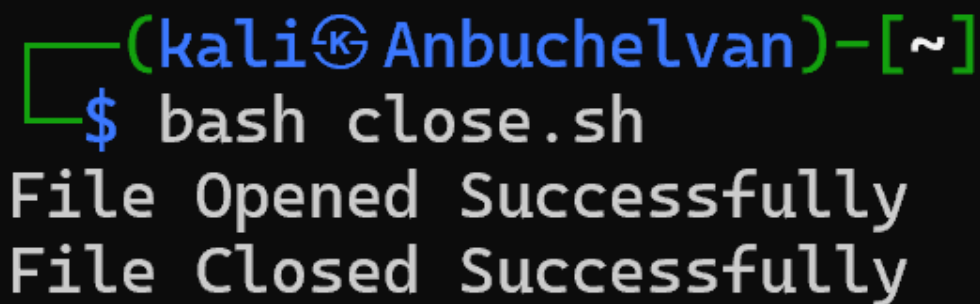
OUTPUT:



```
(kaliⓀ Anbuchelvan)~  
$ nano close.sh
```



```
(kaliⓀ Anbuchelvan)~  
$ chmod +x close.sh
```



```
(kaliⓀ Anbuchelvan)~  
$ bash close.sh  
File Opened Successfully  
File Closed Successfully
```